一、数据通路的概念与地位

定义：数据通路是 CPU 内部数据流动的路径，由==操作元件、状态元件以及它们之间的连线（内部总线）==共同构成，指令执行时，数据在控制信号的控制下沿这条路径流动并被加工

数据通路

- **数据通路是指数据在各功能部件之间传送的路径。**
  - **CPU内部的数据通路是指运算器与各寄存器之间的数据传送路径。**
- **数据在数据通路中的传送操作需要在控制信号的控制下进行。**
- **不同指令在指令周期的不同阶段可能使用不同的数据通路。**
- **同一指令在指令周期的不同阶段也可能使用不同的数据通路。**
- **综上所述，数据通路的结构对CPU内各种数据的传送路径、各种指令执行所需要的微操作（最基本的、不可再分割的动作）控制信号以及控制器的设计等都有重要影响。**
- **数据通路的基本结构可分为：**
  - **总线**
  - **专用通路**

与控制器的分工：

	数据通路	控制器
职责	数据的存储、传送、运算	产生控制信号，规定各部件的动作时序
是否处理数据	是	否，只输出电平信号
组成	操作元件 + 状态元件 + 连线	指令译码器、时序产生器、控制信号生成逻辑
实现方式	组合逻辑 + 时序逻辑	硬布线（组合逻辑/状态机）或微程序（控制存储器）

一句话：控制器给出"做什么、什么时候做"，数据通路负责"实际做"。二者通过一组控制信号（如 `RegWrite`、`MemRead`、`ALUSrc`、`ALUOp`）耦合

与指令执行过程的关系：一条指令的执行划分为**取指周期、间址周期、执行周期、中断周期**四个机器周期，每个机器周期又分为若干**节拍（CPU 时钟周期）**，每个节拍发出一组微操作。数据通路正是这些微操作的物理承载体——**微操作 = 控制信号 + 数据通路上的具体动作**

二、数据通路的组成：操作元件与状态元件

判别标准只有一条：**是否具有记忆功能（是否由触发器构成、是否随时钟沿更新状态）**

1. 操作元件（组合逻辑元件）

特点：**无记忆**，输出仅由当前输入决定，没有时钟；其延迟直接计入关键路径，**决定时钟周期的下限**

元件	功能	要点
ALU（算术逻辑单元）	加、减、与、或、异或、比较等	数据通路核心；输出运算结果 + 状态标志（ZF、CF、OF、SF）；在多周期方案中被**复用**于算术运算、地址计算、分支比较
移位器	算术移位、逻辑移位、循环移位	用于乘除 2 的幂、数据对齐；桶形移位器可一次任意位移
加法器	PC+1 / PC+4、分支目标地址计算	通常**独立于 ALU**，这样可在译码阶段就算出转移地址，缩短控制冒险
指令译码器（ID）	操作码 → 控制信号/微操作命令	逻辑上属控制器，物理上紧邻寄存器组
多路选择器（MUX）	从多个来源中选择一个输出	**数据通路"灵活性"的来源**：凡"不同指令走不同路"处皆有 MUX，如 ALU 第二操作数选寄存器值还是立即数
三态门 / 缓冲器	控制部件是否挂上内部总线	高阻态 = 让出总线，解决总线冲突
符号扩展器 / 零扩展器	16 位立即数扩展为 32 位	有符号数用符号扩展，无符号（地址计算）用零扩展
比较器、编码器	大小比较、优先级编码	编码器常用于中断优先级判别

2. 状态元件（时序逻辑元件）

特点：**有记忆**，由触发器构成，在时钟沿锁存数据；**它们的划分位置就是时钟边界的划分位置**

元件	功能
PC（程序计数器）	存放下一条指令地址；顺序执行时自动加 1/加 4，转移时由 MUX 装入目标地址
IR（指令寄存器）	暂存当前取出的指令，供译码器使用
MAR（存储器地址寄存器）	存放要访问的主存地址
MDR / MBR（存储器数据寄存器）	暂存读出或待写入主存的数据
通用寄存器组 / 寄存器堆（RF）	存放操作数与结果；单周期与流水线方案需 **2 读 1 写**端口，多周期 1 读 1 写即可
PSW / FR（标志寄存器、状态字寄存器）	保存 ALU 产生的 ZF、CF、OF、SF 及中断允许、管态目态等标志，供条件转移使用
暂存器（DR1、DR2、DR3、TMP 等）	多周期方案中暂存中间结果，如 ALU 的两个源操作数 Y、Z
流水线寄存器（段间寄存器）	IF/ID、ID/EX、EX/MEM、MEM/WB；**既传数据也传该指令的控制信号**
其他	中断请求/屏蔽寄存器、总线请求寄存器等

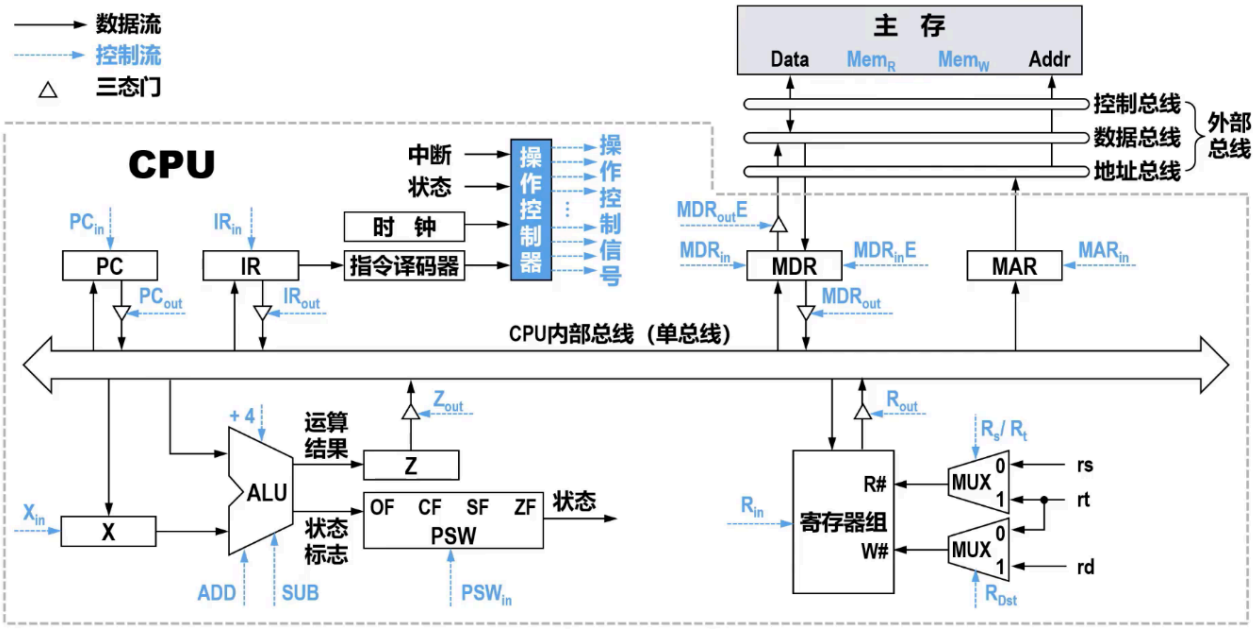
易错提醒：ALU 产生的标志位是"操作元件算出、状态元件保存"——ALU 本身不保存状态，必须写入 PSW 才能在下一拍被分支指令读取

三、数据通路的结构：总线型与专用通路型

1. 总线型结构（内部总线）

所有部件挂在一条（或少数几条）内部总线上，通过==三态门==分时占用

- **单总线结构**：一条内部总线连接 ALU、寄存器组、MAR、MDR、PC 等。同一时刻只能传一组数据，一次微操作通常形如 `R1 → 总线 → ALU 输入`
  - 优点：结构简单、布线少、扩展容易、成本低
  - 缺点：总线成为瓶颈，**同一节拍内不能有两个数据传送**，因此多周期方案中"读寄存器 + 送 ALU"往往要拆成两拍



- **双总线 / 三总线结构**：把 ALU 的两个源操作数分走两条总线，或把地址总线与数据总线分离，使一拍内可并行传送两组数据，减少节拍数
- **片内多总线 / 交叉开关（Crossbar）**：允许多对端口同时通信，并行度高，代价是布线复杂、面积大，现代 CPU 与多核 SoC 采用

关键结论：总线型结构的性能上限受**总线宽度**和**总线仲裁/分时**限制；结构冒险（争用同一部件或同一总线）在总线型结构中更常见

2. 专用通路型结构（点对点直接连接）

部件之间用**固定的专用连线**直接相连，不经过共享总线

- 典型：流水线数据通路中，ALU 输出 → 下一段寄存器 → 再下一段，段间都是专用通路；**数据旁路（Forwarding）网络**也是典型的专用通路，把 EX/MEM、MEM/WB 的结果直接送回到 ALU 输入端
- 优点：无仲裁、无冲突，一拍内可完成多组数据传送，延迟低、时序好控制
- 缺点：连线数量随部件数快速增长（近似平方级），面积和布线代价大，灵活性差（增加新部件需重新布线）

3. 两者的实际关系

真实 CPU 是**混合结构**：主流水路径（取指→译码→执行→访存→写回）用专用通路保证速度，寄存器组到各功能单元、调试与配置通路等低频路径用总线或交叉开关。判别要点是：**追求高吞吐、低延迟的路径用专用通路；追求灵活、低成本、低频使用的路径用总线**

四、数据通路的三种组织形态

核心比较维度是：**硬件是否复用、指令是否重叠、CPI、时钟周期、控制复杂度**

方案	硬件复用	指令重叠	CPI	时钟周期	控制器
单周期	否（部件独占）	否	恒为 1	最长（整条路径）	组合逻辑，最简单
多周期	是（分时复用）	否	平均 4~5	较短	状态机 / 微程序
流水线	是（空间分段）	是	≈1	最短（单段）	较复杂，需处理冒险

1. 单周期数据通路

一条指令的全部操作在一个时钟周期内完成，所有指令周期长度相同

- **时钟周期由最长路径决定**：对 MIPS 而言是 `lw`（取指 → 读寄存器 → ALU 算地址 → 读数据存储器 → 写回），其余指令（如 `add` 不访存）在此周期内大量空转
- **部件不复用**：ALU、存储器各一份且被独占，硬件利用率低
- **控制最简单**：只需一张由操作码和标志位决定的组合逻辑真值表，无需状态机
- **结论**：CPI=1 但频率上不去，工程上不采用，仅作分析基准；也难以容纳乘法、除法等长延迟指令

2. 多周期数据通路

只保留一套硬件，把指令执行划分为若干节拍，每节拍完成一组微操作，由时序产生器（状态机或微程序）控制

典型节拍划分（对应四个机器周期）：

节拍	微操作要点
取指	MAR←PC；读主存；MDR→IR；PC←PC+1(4)
间址（可选）	若为间接寻址，MDR→MAR，再读一次主存取有效地址
执行	ALU 运算 / 算访存地址 / 比较；Load、Store 在此后访问主存
中断（可选）	判中断请求，响应则保护断点、转中断服务程序

- **优点**：硬件省（一套 ALU、单端口存储器、少端口寄存器组即可），时钟周期短、频率高，可支持复杂指令
- **缺点**：CPI 大，简单指令也要多拍；需要大量暂存器（DR1/DR2/DR3）保存中间结果；控制逻辑复杂、验证困难
- **典型代表**：早期 CISC 处理器（8086 一类）

3. 流水线数据通路

在多周期分段的基础上，**段间插入流水线寄存器**，切断长组合路径，使多条指令的不同阶段在同一时刻并行执行

- **性能**：k 段流水线执行 n 条指令，总时间 ≈ $(k-1+n)\times\tau$；n 很大时 CPI→1，吞吐提升接近 k 倍
- **必须明确**：流水线提高的是**吞吐率**，**单条指令的延迟（latency）反而变长**
- **流水线寄存器要传控制信号**：后级执行时需要该指令的 `RegWrite`、`MemRead`、`ALUSrc` 等，这些信号必须随指令一起下传，这是常考细节
- **三类冒险及 408 要求的处理办法**：

冒险类型	成因	处理办法
结构冒险	两条指令同拍争用同一部件（如指令与数据共用存储器）	资源重复配置（分离指令 Cache 与数据 Cache）、插入气泡、资源分时
数据冒险	后一条指令需使用前一条尚未写回的结果（写后读 RAW）	**数据旁路/转发（Forwarding）**为主；无法转发时**插入气泡/暂停（Stall）**；编译优化（指令调度）减少相关
控制冒险	转移指令使后续取到的指令可能作废	分支预测（静态/动态）、延迟槽、把分支判断提前到 ID 段

其中 **load-use 冒险**（紧跟的指令要用刚 load 的数据）即使有转发也通常必须停顿 1 个周期，是经典考点

三者关系

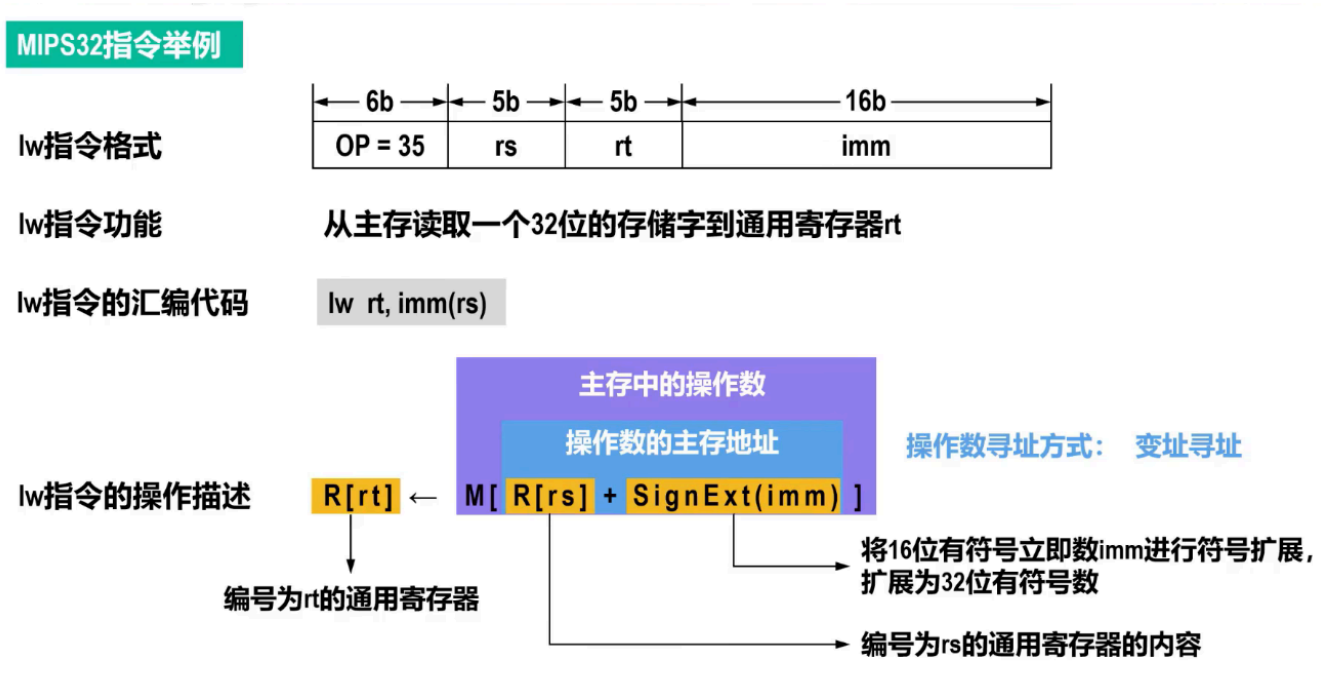
单周期回答"**一条指令需要哪些元件、数据怎么走**"；多周期回答"**这些元件如何分时复用**"（用时间换硬件）；流水线回答"**如何让多条指令在时间上重叠**"（用流水线寄存器换吞吐）。而三者共同的设计手法只有一个：**找出不同指令数据流的差异，用 MUX 收敛成一套统一硬件，再由控制信号选择路径**

常考结论速记：

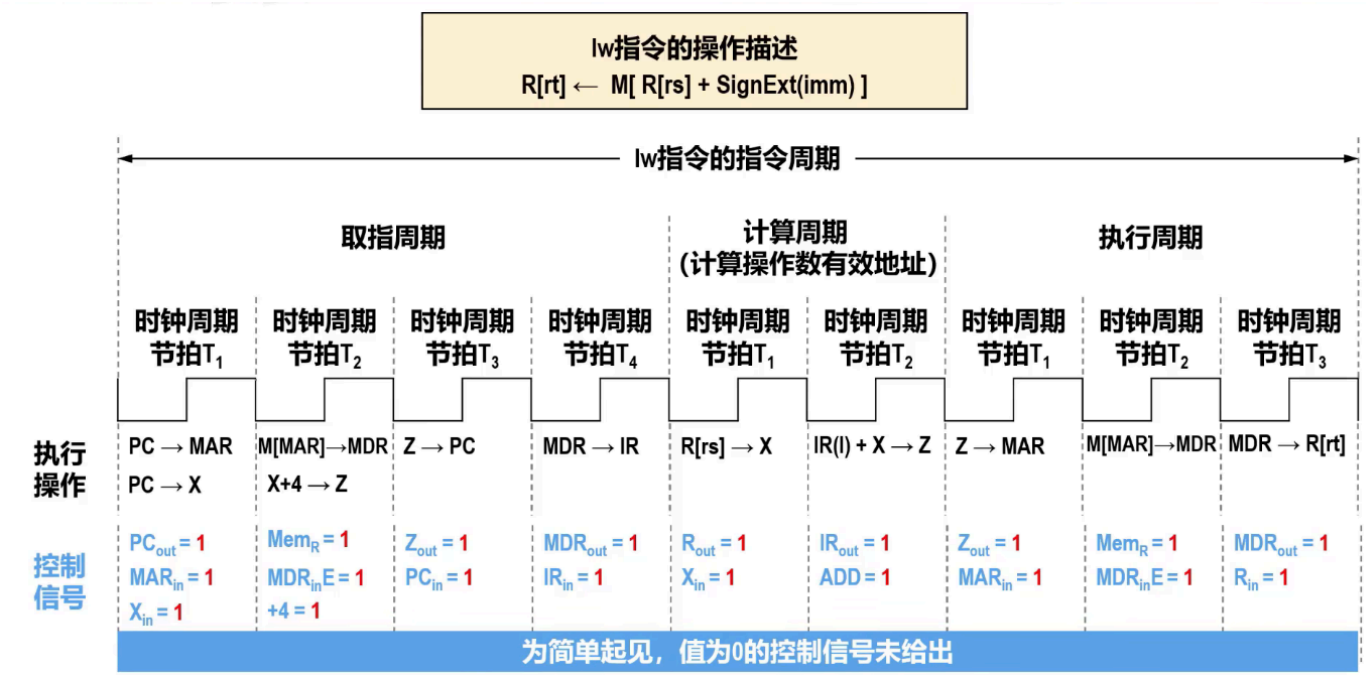
- **数据通路 = 操作元件 + 状态元件 + 连线**
- 操作元件是组合逻辑、决定关键路径，状态元件是时序逻辑、决定时钟边界
- 单周期 CPI=1 但周期最长；多周期靠复用省硬件、CPI 大；流水线 CPI≈1、吞吐高，代价是三类冒险

五、以MIPS32指令举例的单总线数据通路

5.1 lw指令

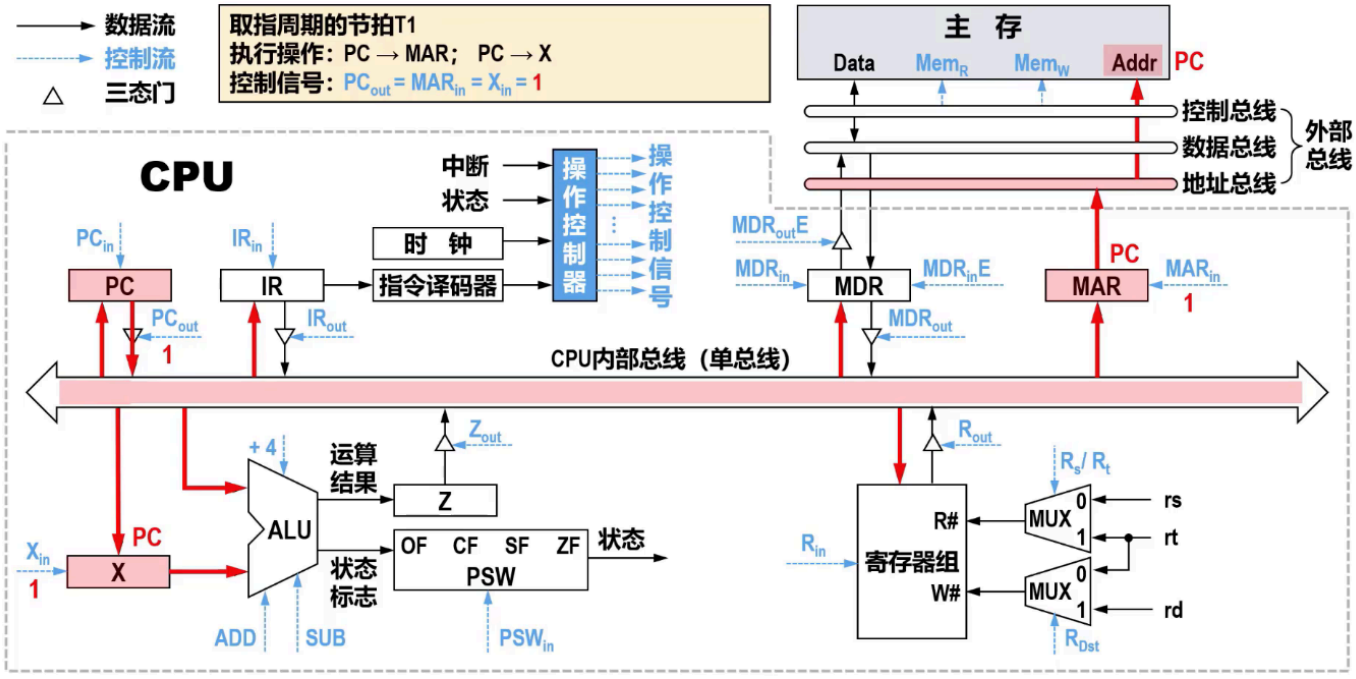


lw指令的指令周期：

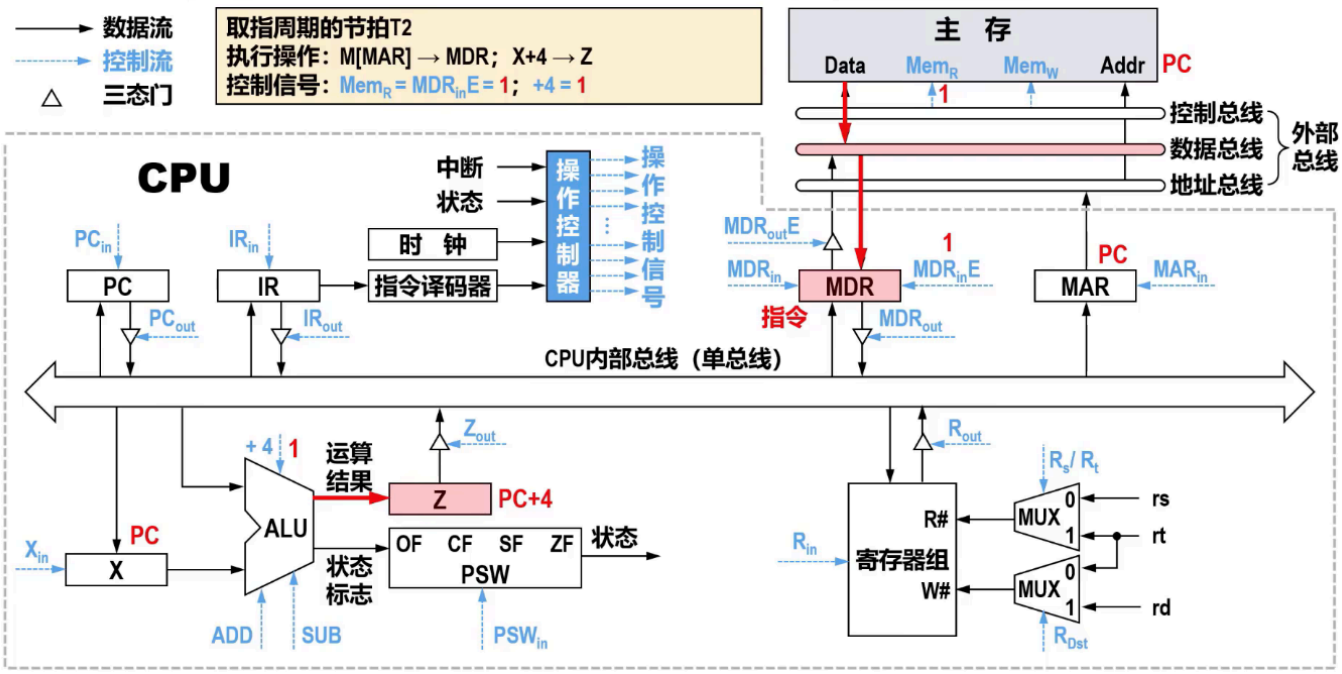


取指周期

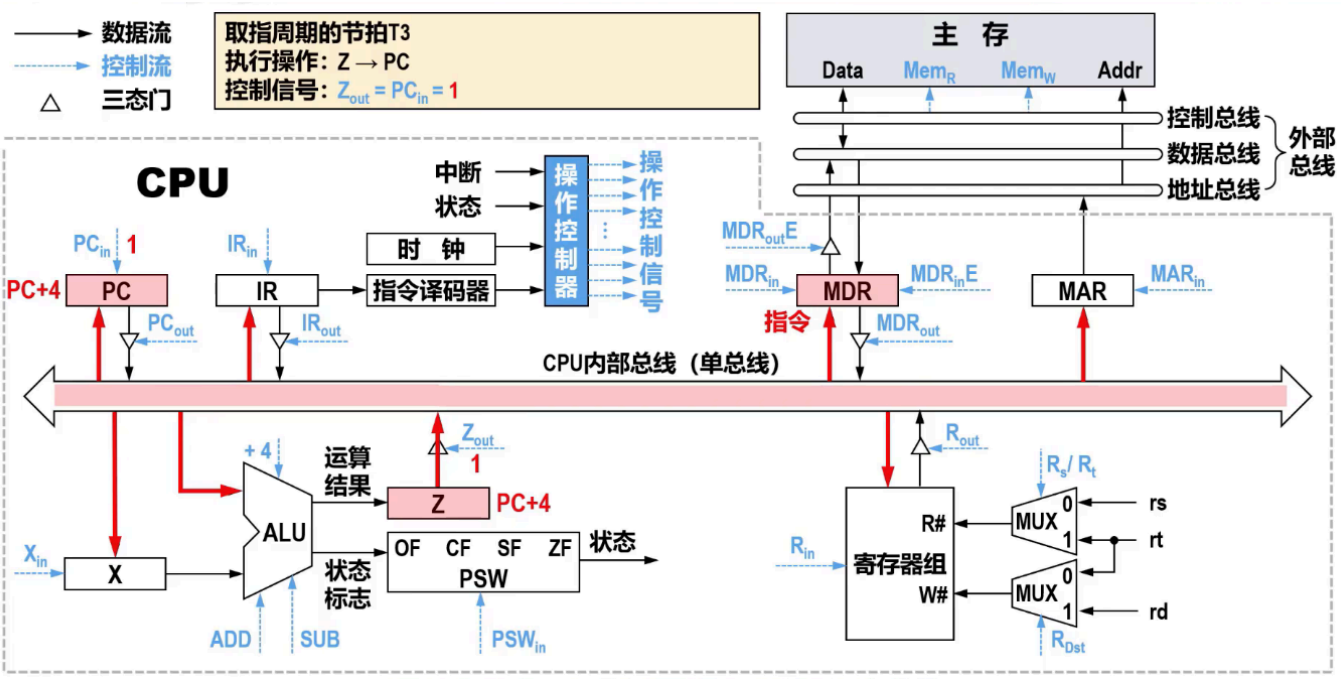
节拍一



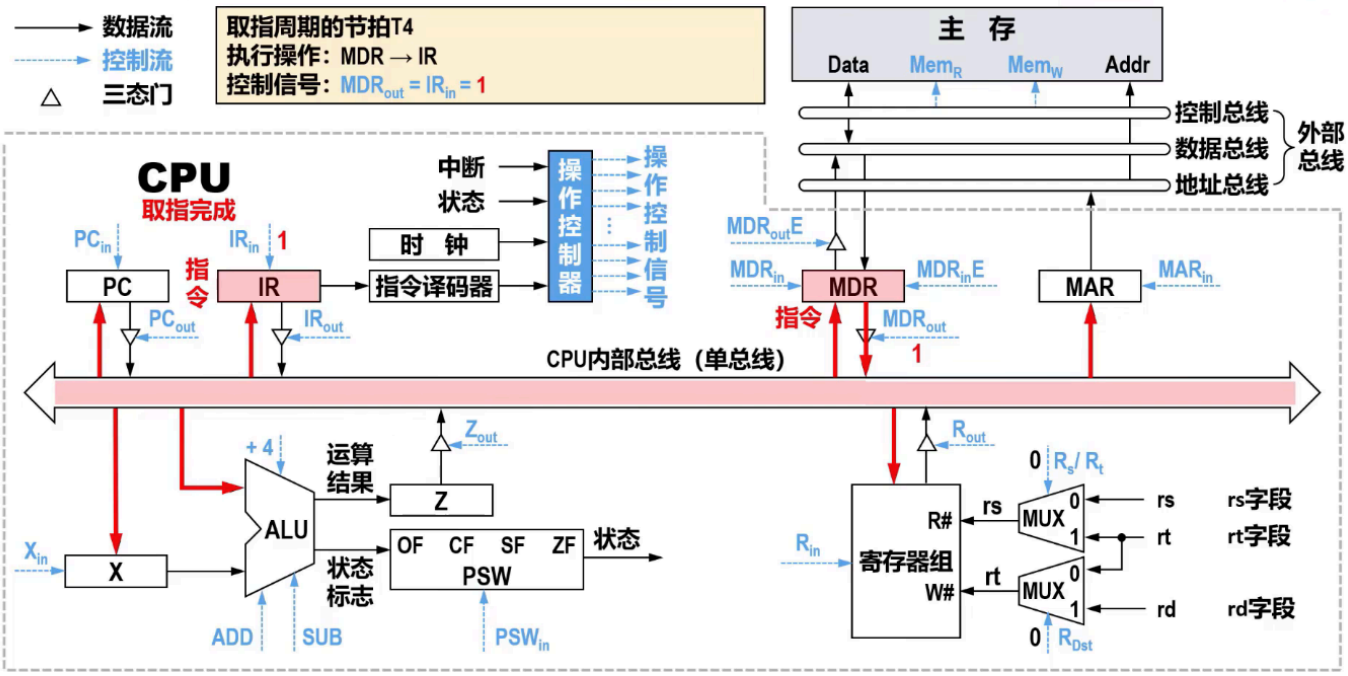
节拍二


数据流
控制流
三态门
取指周期的节拍T2
执行操作：M[MAR] → MDR；X+4 → Z
控制信号：MemR = MDRinE = 1；+4 = 1
主 存
Data
MemR
MemW
Addr
PC
控制总线
数据总线
地址总线
外部总线
CPU
中断
状态
时 钟
操作控制器
操作控制信号
PCin
IRin
PC
IR
指令译码器
PCout
IRout
MDRoutE
MDRin
MDR
MDRinE
MAR
MARin
指令
MDRout
CPU内部总线（单总线）
+4
1
运算结果
Zout
Z
PC+4
Rout
Rs / Rt
Xin
X
ALU
状态标志
OF CF SF ZF
PSW
状态
Rin
寄存器组
R#
W#
MUX
rs
rt
rd
ADD
SUB
PSWin
RDst

节拍三

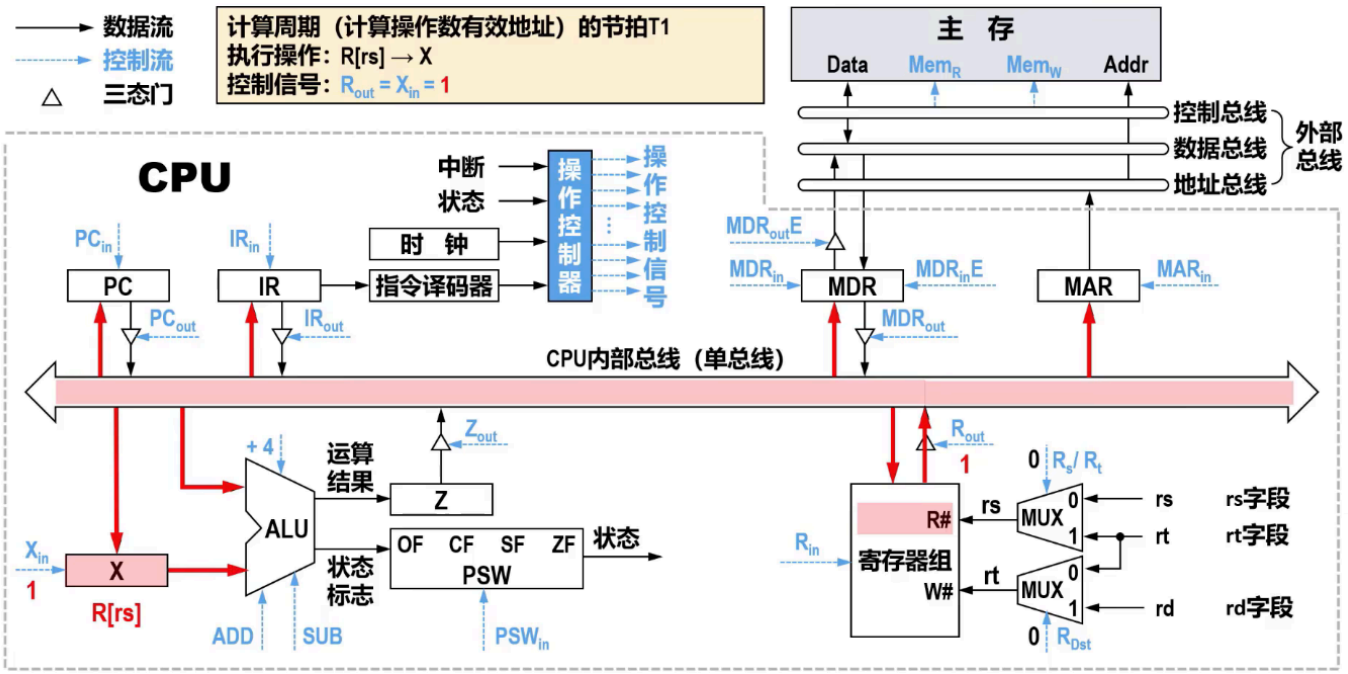

数据流
控制流
三态门
取指周期的节拍T3
执行操作：Z → PC
控制信号：Zout = PCin = 1
主 存
Data
MemR
MemW
Addr
控制总线
数据总线
地址总线
外部总线
CPU
中断
状态
时 钟
操作控制器
操作控制信号
PCin
1
PC+4
PC
IR
指令译码器
PCout
IRout
MDRoutE
MDRin
MDR
MDRinE
MAR
MARin
指令
MDRout
CPU内部总线（单总线）
+4
运算结果
Zout
1
Z
PC+4
Rout
Rs / Rt
Xin
X
ALU
状态标志
OF CF SF ZF
PSW
状态
Rin
寄存器组
R#
W#
MUX
rs
rt
rd
ADD
SUB
PSWin
RDst

节拍四

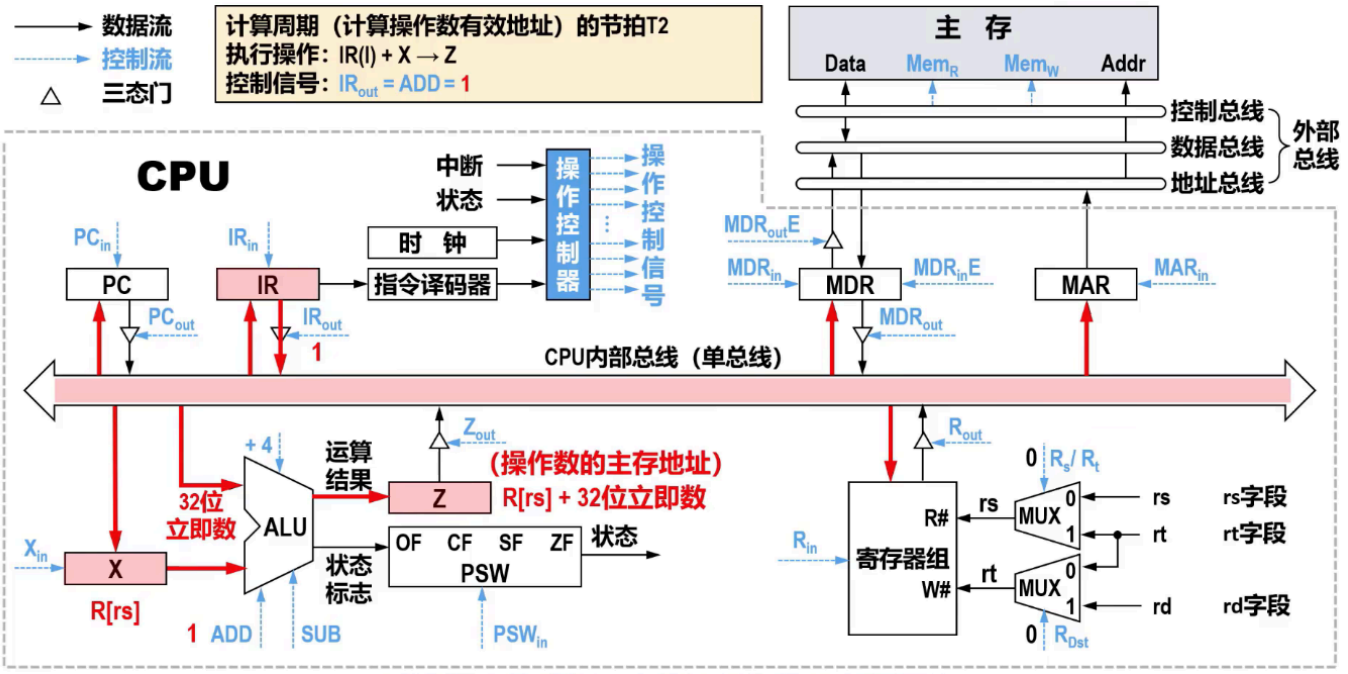

数据流
控制流
三态门
取指周期的节拍T4
执行操作：MDR → IR
控制信号：MDR_out = IR_in = 1
主 存
Data
Mem_R
Mem_W
Addr
控制总线
数据总线
地址总线
外部总线
CPU
取指完成
中断
状态
时 钟
操作控制器
操作控制信号
PC_in
IR_in
1
指令
PC
IR
指令译码器
PC_out
IR_out
MDR_out E
MDR_in
MDR
MDR_in E
MAR
MAR_in
指令
MDR_out
1
CPU内部总线（单总线）
+4
运算结果
Z_out
Z
R_out
0
R_s / R_t
X_in
X
ALU
状态标志
OF CF SF ZF
PSW
状态
R_in
寄存器组
R#
W#
rs
rt
MUX
rs
rt
rd
rs字段
rt字段
rd字段
ADD
SUB
PSW_in
R_Dst

计算周期

节拍一

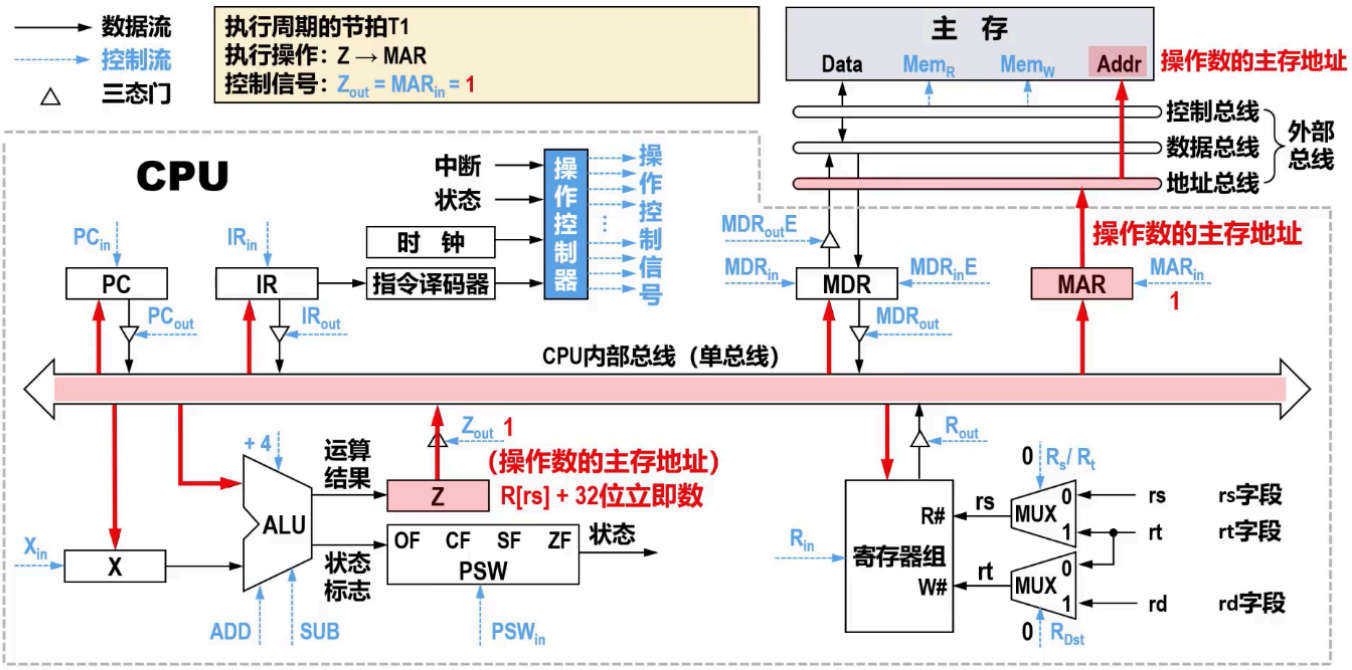

数据流
控制流
三态门
计算周期（计算操作数有效地址）的节拍T1
执行操作：R[rs] → X
控制信号：R_out = X_in = 1
主 存
Data
Mem_R
Mem_W
Addr
控制总线
数据总线
地址总线
外部总线
CPU
中断
状态
时 钟
操作控制器
操作控制信号
PC_in
IR_in
PC
IR
指令译码器
PC_out
IR_out
MDR_out E
MDR_in
MDR
MDR_in E
MAR
MAR_in
MDR_out
CPU内部总线（单总线）
+4
运算结果
Z_out
Z
R_out
1
0
R_s / R_t
X_in
1
X
R[rs]
ALU
状态标志
OF CF SF ZF
PSW
状态
R_in
寄存器组
R#
W#
rs
rt
MUX
rs
rt
rd
rs字段
rt字段
rd字段
ADD
SUB
PSW_in
R_Dst

节拍二

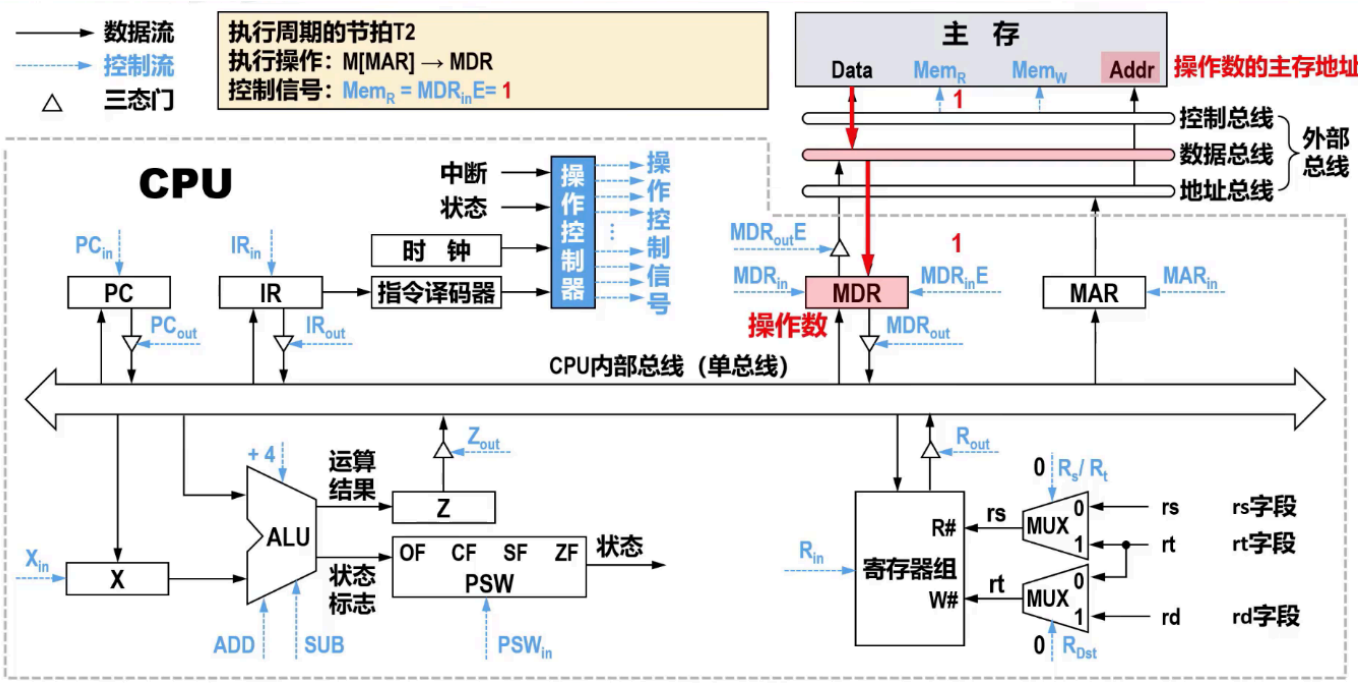

数据流
控制流
三态门
计算周期（计算操作数有效地址）的节拍T2
执行操作：IR(l) + X → Z
控制信号：IR_out = ADD = 1
主 存
Data
Mem_R
Mem_W
Addr
控制总线
数据总线
地址总线
外部总线
CPU
中断
状态
时 钟
操作控制器
操作控制信号
PC_in
IR_in
PC
IR
指令译码器
PC_out
IR_out
1
MDR_outE
MDR_in
MDR
MDR_inE
MDR_out
MAR
MAR_in
CPU内部总线（单总线）
Z_out
+4
运算结果
(操作数的主存地址)
Z
R[rs] + 32位立即数
32位立即数
ALU
X_in
X
R[rs]
OF CF SF ZF
PSW
状态
状态标志
1 ADD
SUB
PSW_in
R_out
0 R_s/R_t
R_in
R#
寄存器组
W#
rs
rt
MUX
0
1
rs
rt
rd
rs字段
rt字段
rd字段
0 R_Dst

执行周期

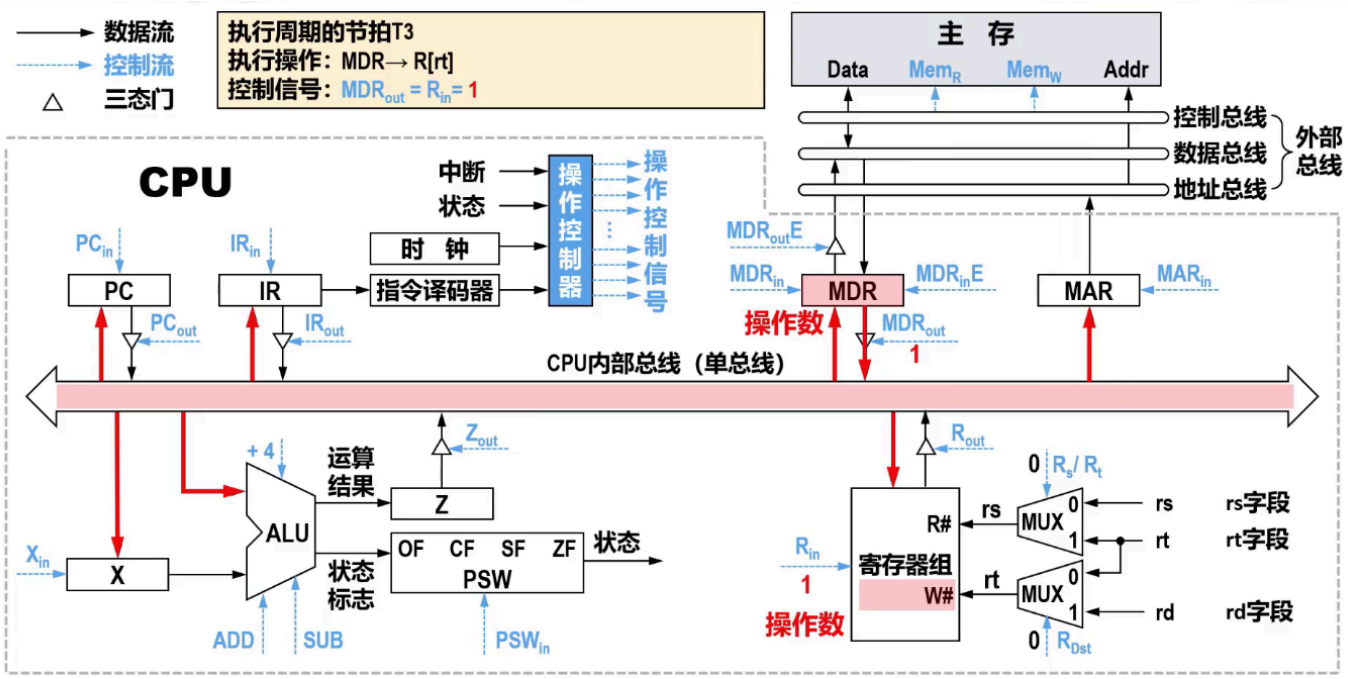
节拍一


数据流
控制流
三态门
执行周期的节拍T1
执行操作：Z → MAR
控制信号：Z_out = MAR_in = 1
主 存
Data
Mem_R
Mem_W
Addr
操作数的主存地址
控制总线
数据总线
地址总线
外部总线
CPU
中断
状态
时 钟
操作控制器
操作控制信号
PC_in
IR_in
PC
IR
指令译码器
PC_out
IR_out
MDR_outE
MDR_in
MDR
MDR_inE
MDR_out
MAR
MAR_in
1
CPU内部总线（单总线）
Z_out 1
+4
运算结果
(操作数的主存地址)
Z
R[rs] + 32位立即数
ALU
X_in
X
OF CF SF ZF
PSW
状态
状态标志
ADD
SUB
PSW_in
R_out
0 R_s/R_t
R_in
R#
寄存器组
W#
rs
rt
MUX
0
1
rs
rt
rd
rs字段
rt字段
rd字段
0 R_Dst

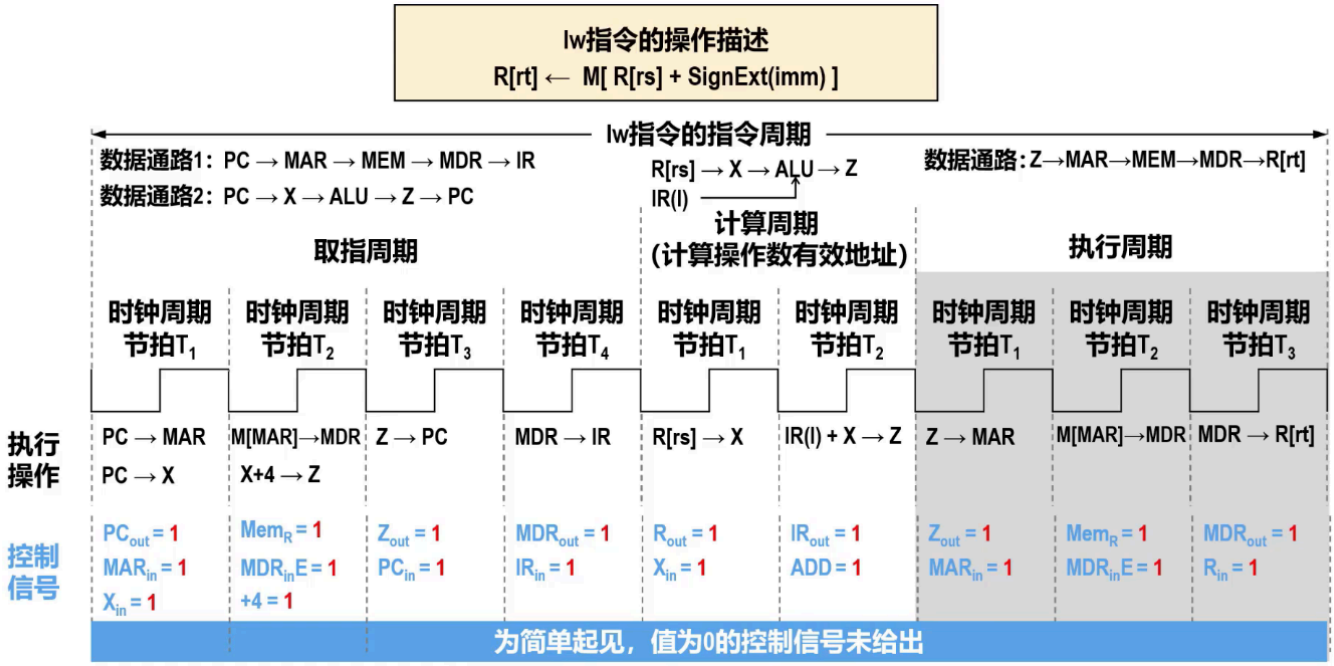
节拍二


数据流
控制流
三态门
执行周期的节拍T2
执行操作：M[MAR] → MDR
控制信号：MemR = MDRinE= 1
主 存
Data
MemR
MemW
Addr
操作数的主存地址
1
控制总线
数据总线
地址总线
外部总线
CPU
中断
状态
时 钟
指令译码器
操作控制器
操作控制信号
PCin
IRin
PC
IR
PCout
IRout
MDRoutE
MDRin
MDR
MDRinE
MAR
MARin
操作数
MDRout
CPU内部总线（单总线）
+ 4
运算结果
Zout
Z
ALU
状态标志
OF CF SF ZF
PSW
状态
Xin
X
ADD
SUB
PSWin
Rout
0
Rs/ Rt
R#
rs
MUX
rt
寄存器组
W#
Rin
rs字段
rt字段
rd字段
RDst

节拍三


数据流
控制流
三态门
执行周期的节拍T3
执行操作：MDR→ R[rt]
控制信号：MDRout = Rin= 1
主 存
Data
MemR
MemW
Addr
控制总线
数据总线
地址总线
外部总线
CPU
中断
状态
时 钟
指令译码器
操作控制器
操作控制信号
PCin
IRin
PC
IR
PCout
IRout
MDRoutE
MDRin
MDR
MDRinE
MAR
MARin
操作数
MDRout
1
CPU内部总线（单总线）
+ 4
运算结果
Zout
Z
ALU
状态标志
OF CF SF ZF
PSW
状态
Xin
X
ADD
SUB
PSWin
Rout
0
Rs/ Rt
R#
rs
MUX
rt
寄存器组
W#
Rin
1
操作数
rs字段
rt字段
rd字段
RDst

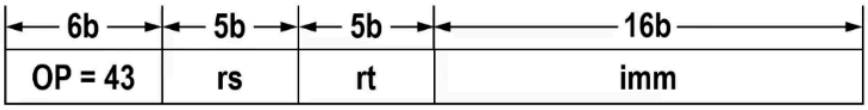
总结



5.2 sw指令

MIPS32指令举例

sw指令格式

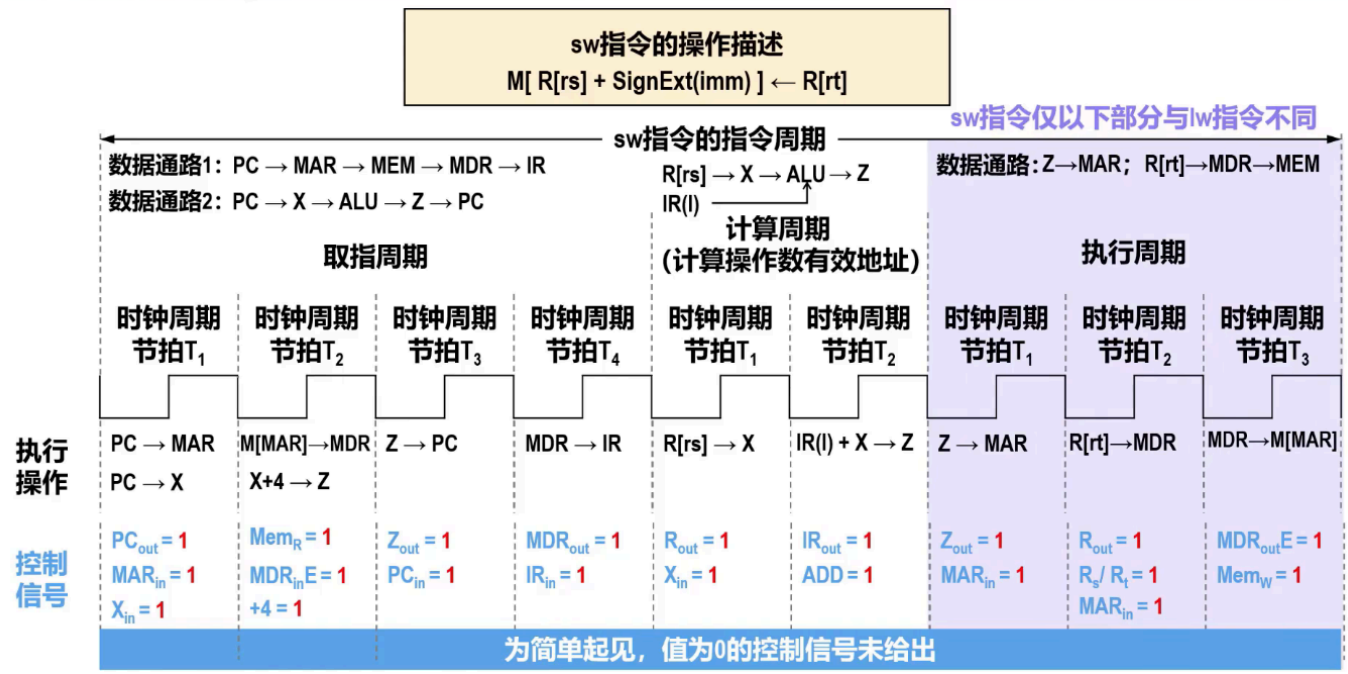


sw指令功能　　将通用寄存器rt的内容（32位）写入指定的主存单元

sw指令的汇编代码　　sw rt, imm(rs)

sw指令的操作描述　　M[R[rs] + SignExt(imm)] ← R[rt]

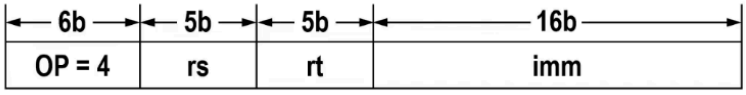
sw指令的各个节拍



5.3 beq指令

MIPS32指令举例

beq指令格式



beq指令功能　比较通用寄存器rs和rt的值，如果相等则进行分支跳转。

beq指令的汇编代码　beq rs, rt, imm

beq指令的操作描述　If (R[rs] == R[rt])　PC ← PC + 4 + SignExt(imm) << 2

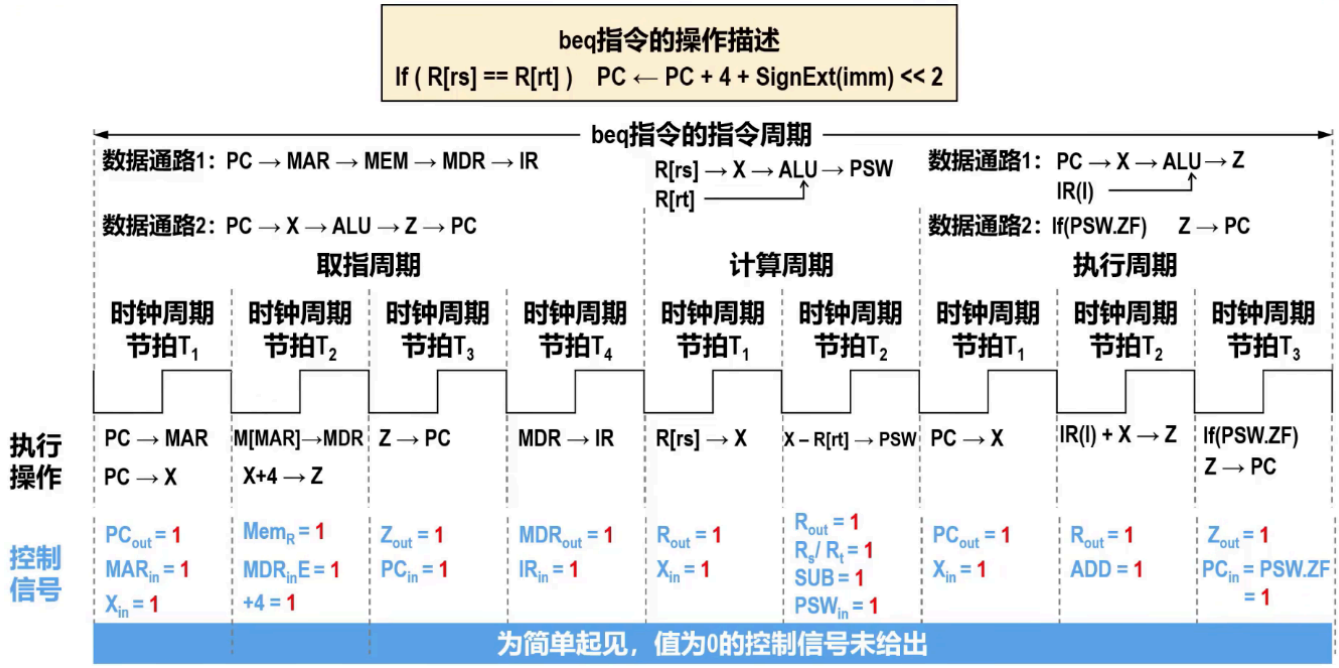
- 立即数imm的值表示分支目标地址相对下一条指令（取指阶段已更新为PC+4）的指令条数

① 将imm进行符号扩展，扩展为32位有符号数。

② 将扩展得到的32位有符号数左移两位（相当于乘以4），目的是计算字节偏移量。

上述操作仅简单的利用IR_{out}信号控制立即数输出到CPU内部总线，相关硬件逻辑并未在图中画出。

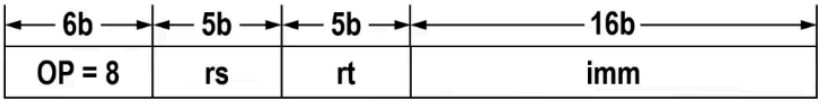
beq指令的各个节拍



5.4 addi指令

MIPS32指令举例

addi指令格式

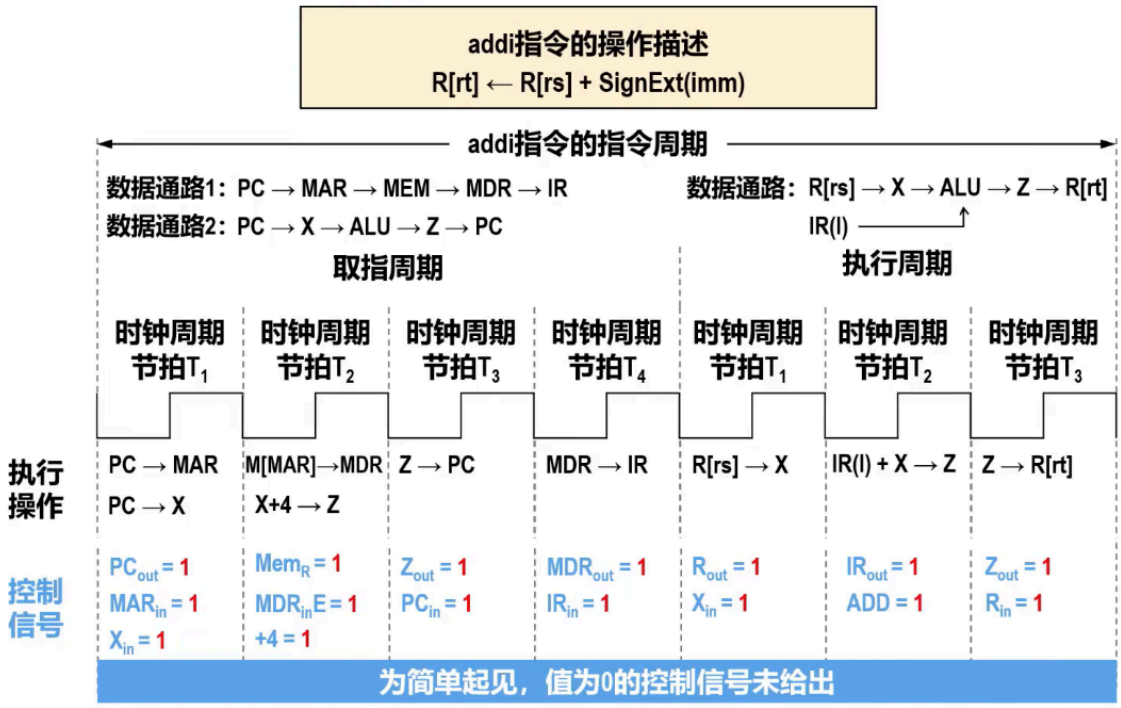


addi指令功能 将通用寄存器rs的内容与立即数imm相加，结果送入通用寄存器rt。

addi指令的汇编代码 addi rt, rs, imm

addi指令的操作描述 R[rt] ← R[rs] + SignExt(imm)

addi指令的各个节拍



5.5 add指令

MIPS32指令举例

add指令格式

6b	5b	5b	5b	5b	6b
OP = 0	rs	rt	rd	shamt	funct=32

shamt为位移量，对于add指令取值总是为0，因此add指令不需要位移操作；

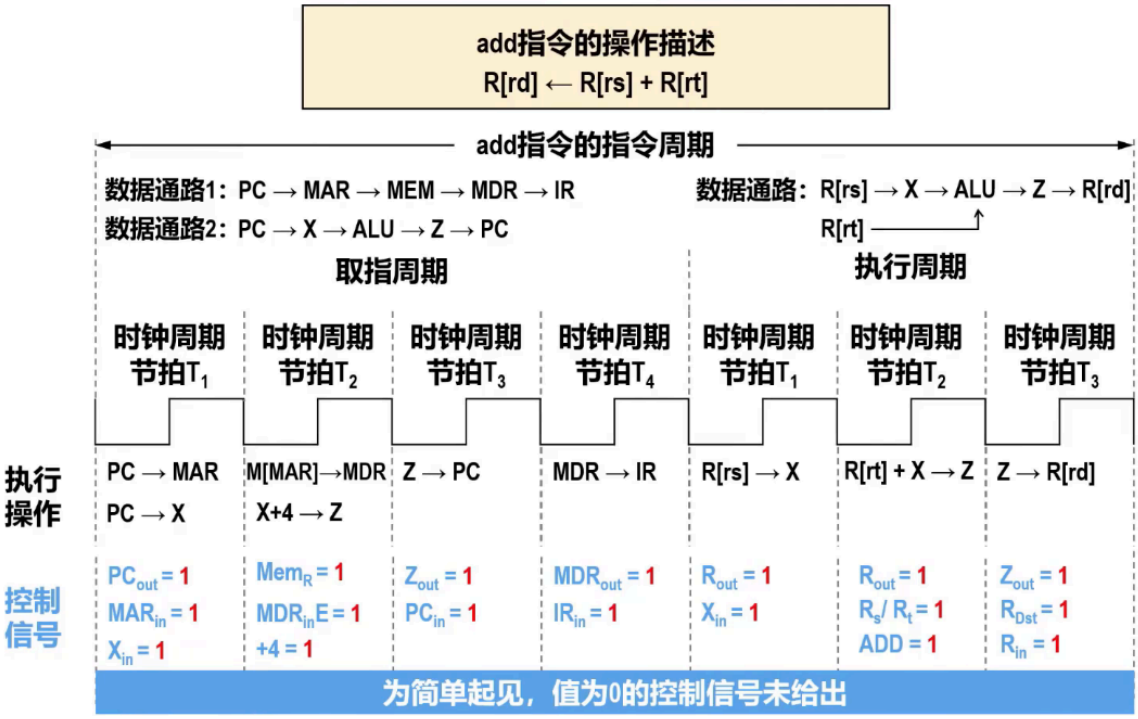
funct功能码，对于add指令取值为32

add指令功能　将通用寄存器rs的内容与通用寄存器rt的内容相加，结果送入通用寄存器rd。

add指令的汇编代码　`add rd, rs, rt`

add指令的操作描述　R[rd] ← R[rs] + R[rt]

add指令的各个节拍



5.6 MIPS32指令对比

